rowdb M1 報告：kill -9 で確定したコミットが消えないかを確かめる

自作の行指向ストレージエンジンに、ランダムなタイミングで kill -9 を当て続け、復旧した後の中身をテスト側の記録と突き合わせた。

2026年11月10日 ／ 実行環境：M6 Mac mini（macOS） ／ 最後のまとめ実行：22:29〜22:34 JST、計1,000回の kill ／ 実装：AI（Claude）、設計と検証方針の判断：神田

概要

確かめたかったことは二つある。一つは、書き込みの途中でプロセスを kill -9 で止めても、コミット完了を返したトランザクションは復旧後に必ず残るか。そして、コミットの途中で止まったトランザクションは全部残るか全部消えるかのどちらかになり、一部だけ残ったり、取り消したものが見えたりしないか。もう一つは、それを確かめるテスト自体が、わざと壊したエンジンの誤りを本当に見つけられるか、である。

最後のまとめ実行では、正常なエンジンの4つの条件・合計660サイクルで行ったチェック459回が、すべて期待どおりの状態だった。コミットの途中（台帳に完了の印を書く前）で殺されたトランザクションは414件あり、どれも「全部反映」か「全部なし」のどちらかで、一部だけ残ったものはなかった。わざと壊した3種類のエンジンは、どれも食い違いとして検出できた（図1）。

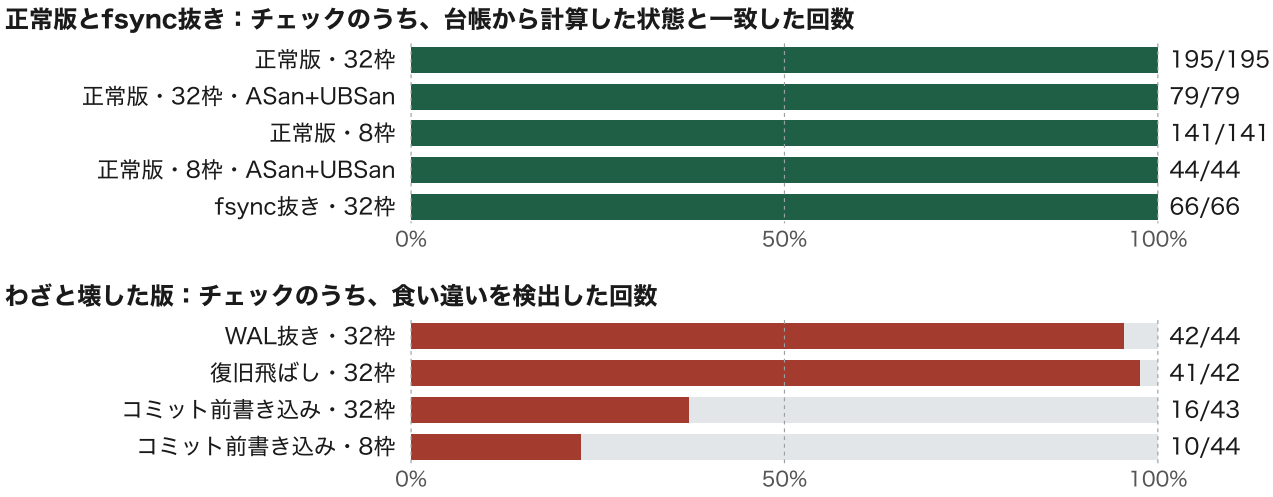


図1　最後のまとめ実行の結果。上は「食い違い0回」が合格条件で、棒は期待どおりだったチェックの割合。下はわざと壊した版で、棒は食い違いを検出したチェックの割合なので、上下の長さは比べられない。チェックはkillのたびに行うわけではない（詳しくは「確かめ方」）。

範囲の限界も先に書いておく。kill -9 ではOSのページキャッシュが残るので、fsyncを呼び忘れる誤りはこのテストでは見えない（fsyncを全部抜いた版も66回のチェックにすべて合格した）。また、このMacの素のfsyncはデータをドライブへ渡すだけで、ドライブ内のキャッシュから記録面へ書くことは保証しない。電源断に耐えるかどうかは、M1では確かめていない。

作ったもの

rowdb は C++20 で書いた行指向のストレージエンジンで、テスト用のワークロードを含めて約1,000行ある。データファイルは4KBのページに分かれ、各ページは先頭にCRC32（中身が壊れていないかを確かめる値）を持つ。行はページの中に入れ、ページの先頭にある表に各行の位置と長さを記録する。削除した場所は使い直さない。メモリ上のページ置き場（Buffer Pool）は既定で32枚分あり、足りなくなると最近使っていないページから追い出す。トランザクションは一度に一つだけ動く。

コミットまでの流れは図2の上段のとおりである。変更したページは、コミットするまでデータファイルに書かない。Buffer Poolが一杯になっても、コミット前のページは追い出さない（この方針をNO-STEALと呼ぶ）。全部の枠がコミット前のページで埋まったらエラーを返し、ワークロードはそのトランザクションを取り消す。コミットでは、変更したページ全体の写しとCOMMIT記録をWAL（データファイルより先に書くログ）にまとめて追記し、fsyncしてから完了を返す。この「完了を返す」ことを、以下ではACKと呼ぶ。

データファイル本体への反映は後回しにする。コミット済みの変更があるページは、Buffer Poolから追い出すときにデータファイルへ書く。コミットでWALが512KBを超えたら、そのcommit()の中でチェックポイントを行う。残りの変更済みのページを全部データファイルに書いてfsyncし、WALを空にしてから、完了を返す。起動時の復旧では、WALに残っているもののうち、COMMIT記録まで揃ったトランザクションのページだけをデータファイルへ書き戻す。COMMIT記録のないページの写しは捨てる。コミット前のページはそもそもデータファイルに書かれないので、復旧時に書き込みを取り消す仕組み（undo）は持っていない。トランザクションを取り消すときは、メモリの上だけで戻す。トランザクションが最初にページを触ったときにそのページの写しを取っておいて書き戻し、新しく確保したページは確保自体を取り消す。

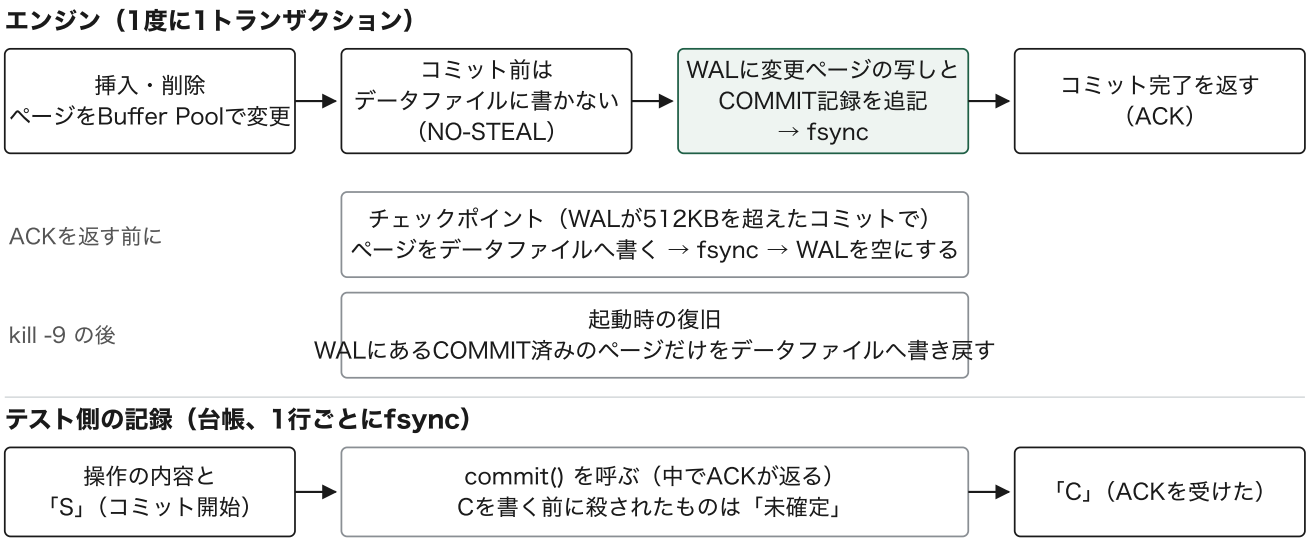


図2　上：エンジンがコミットするまでの流れと、後で行うチェックポイント、kill -9 の後の復旧。下：テスト側がcommit() の前後に台帳へ書くもの。台帳は1行ごとにfsyncするので、kill -9 の後も残る。

確かめ方

テスト用のワークロードは、1〜12個の挿入・削除からなるトランザクションを繰り返す。Buffer Poolに収まらなくなったトランザクションは取り消し、それ以外は5%の確率で取り消して、残りをコミットする。取り消した回数は数えていない。行数は1,500行前後になるようにしてある。コミットする前に、そのトランザクションの操作の内容と「S」を台帳に書いてfsyncし、ACKを受けたら「C」を書く（図2の下段）。

テストのハーネスは、ワークロードを起動してから0〜250msの一様乱数だけ待ち、SIGKILLを送る。ただし、このMacではPythonの待ち時間が指定より長くなり、実際にkillした時刻は正常版・32枠の300サイクルで中央値206ms、90%点310ms、最大392msだった。killの後に、復旧してから全行を出力するコマンドを走らせ、その中身を台帳から計算した期待状態と比べる。このチェックはkillのたびに70%の確率（ハーネスの既定値）で行う。ただし最後のサイクルと、SIGKILL以外で終わったサイクルは必ず行う。チェックを行わなかった分は、次のチェックのときの状態でまとめて確かめるので、途中で現れて消えた行は見えない。期待状態の条件は三つある。Cまで書かれたトランザクションは全部の操作が反映されていること。Sだけが書かれたトランザクション（「未確定」のもの。1サイクルに最大1件）は、全部反映されているか全部ないかのどちらかであること。それ以外の行は一つもないこと。未確定には、commit() の途中で殺されたものだけでなく、ACKが返った後、Cを書く前に殺されたものも入る。そのため、「ACKしたものは必ず残る」を厳密に確かめているのは、台帳にCまで書けたトランザクションについてである。

DBはサイクルをまたいで使い続ける。復旧した状態の上にまた書き込んで、また殺す、を繰り返すので、クラッシュが積み重なっても壊れないかを見ることになる。わざと壊した版で食い違いが出たときは、そのDBを消して空から始め直す。

表1　最後のまとめ実行の9本の条件

実行	エンジンとビルド	枠数とチェックポイント	サイクル	合格条件
正常版・32枠	正常なエンジン、最適化ビルド	32枚／512KB	300	食い違い0回
正常版・32枠・ASan+UBSan	正常なエンジン、メモリ誤りと未定義動作の検査つき	32枚／512KB	100	食い違い0回
正常版・8枠	正常なエンジン、最適化ビルド	8枚／64KB	200	食い違い0回
正常版・8枠・ASan+UBSan	正常なエンジン、検査つき	8枚／64KB	60	食い違い0回
fsync抜き・32枠	fsyncを全部飛ばす。kill -9 では正常版と同じになるはず	32枚／512KB	100	食い違い0回
WAL抜き・32枠	コミットでWALを書かない	32枚／512KB	60	1回以上検出
復旧飛ばし・32枠	起動時にWALを読まずに捨てる	32枚／512KB	60	1回以上検出
コミット前書き込み・32枠	変更したページをすぐデータファイルにも書く（NO-STEAL違反）	32枚／512KB	60	1回以上検出
コミット前書き込み・8枠	同上	8枚／64KB	60	1回以上検出

まとめ実行は、どれか1本でも合格条件を満たさなければ失敗で終わる。正常版とfsync抜きでは、killする前にワークロードが終わってしまった回があっても失敗にした。正常版とfsync抜きでは、中身が薄すぎる場合も失敗にした。具体的には、チェックが0回、ACK済みのコミットが0件、コミット中のkillが0回、トランザクションの実行中に当たったkillがサイクル数の半分未満、のどれかである。この「半分」という線は、それまでの実行結果を見てから決めた目安で、実験の前に決めていた基準ではない。検査つきのビルドは、問題を報告した時点でプロセスを止める設定で走らせた。

結果

表2　最後のまとめ実行の結果（22:29〜22:34 JST、9本とも合格条件を満たした）

実行	サイクル	チェック	一致／検出	コミット中のkill（反映・未反映）	ACK済み	読み込み前のkill
正常版・32枠	300	195	一致 195	176（72・104）	163,695	27
正常版・32枠・ASan+UBSan	100	79	一致 79	65（14・51）	25,883	5
正常版・8枠	200	141	一致 141	133（73・60）	97,182	13
正常版・8枠・ASan+UBSan	60	44	一致 44	40（19・21）	14,703	2
fsync抜き・32枠	100	66	一致 66	58（10・48）	75,009	5
WAL抜き・32枠	60	44	検出 42	31（2・0）	92,789	0
復旧飛ばし・32枠	60	42	検出 41	47（1・0）	31,509	0
コミット前書き込み・32枠	60	43	検出 16	39（13・4）	19,318	2
コミット前書き込み・8枠	60	44	検出 10	38（24・2）	14,094	1

「コミット中のkill」は、Sが書かれてCが書かれる前に殺された未確定のトランザクションの数で、括弧内は、期待どおりだったチェックの中で、復旧後に全部反映されていた数と全部なかった数。壊れ版では、食い違いが出たチェックに含まれる未確定分を分けていないので、括弧内の合計は前の数と合わない。「読み込み前のkill」は、起動時の全行読み込みが終わる前に殺された回数。壊れ版は食い違いを検出するたびにDBを作り直すので、ACK済みの数は正常版と比べられない。

正常版の4本とfsync抜きは、チェックがすべて期待どおりだった。正常版4本で、ACK済みのトランザクションは合計301,463件ある。killする前にワークロードが終わった回は、9本を通して0回だった。

WAL抜きと復旧飛ばしは、ほとんどのチェックで食い違いを検出した。見逃したのはWAL抜きで2回、復旧飛ばしで1回である。最初のまとめ実行で同じような見逃しを調べたときは、どれもkillがチェックポイントの直後に当たり、データファイルへの書き込みがもう終わっていたので、壊れた版でも失うデータがなかった。今回の見逃しは一つずつは確かめていない。

コミット前書き込み版の検出は、32枠で43回中16回、8枠で44回中10回と少ない。理由はまだ確かめていない。この版の誤りが表に出るには、コミット前の変更がデータファイルに書かれた後、トランザクションが終わる前に殺され、しかもその変更が復旧の書き戻しで上書きされない必要がある、とコードからは考えている。ほかに、killがfsyncを待っている時間に当たりやすく、トランザクションの最中には当たりにくいからではないか、という見方もある。どちらの影響が大きいのかは切り分けていない。検出した回の中身は、32枠の16回では、余分な行がある回が7回、ACK済みの挿入が欠けている回が8回（う

ち7回は、1つのトランザクションの挿入が一部だけ残っていた)、ACK済みの挿入以外の行が欠けている回が1回、台帳との矛盾が3回だった（1回に複数が重なることがある)。台帳との矛盾とは、ワークロードが復旧後のDBから読んだ行をもとに行った操作が、台帳から計算した期待状態と食い違った回である。たとえば、期待状態にはない行を削除していた。8枠の10回では、余分な行がある回は1回だけで、ACK済みの挿入が欠けている回が6回（6回とも一部だけ残っていた)、それ以外の行が欠けている回が2回、台帳との矛盾が2回だった。行が欠けるのは、コミットしていない削除がデータファイルに書かれたまま殺されたからだと考えている。これもNO-STEAL違反の症状だが、一つずつは確かめていない。

このテストには、続けるほど中身が薄くなる性質もある。図3は正常版・32枠の300サイクルについて、起動時の全行読み込みが終わった時刻とkillの時刻を並べたものである。削除した場所を使い直さないのでデータファイルは伸び続け（最後は約63MB)、読み込みは最初の10サイクルでは長くても8.6msだったのが、最後の50サイクルでは中央値117msになった。読み込みが終わる前に当たったkillは、前半の150サイクルで10回、後半で17回あった。その分だけ、トランザクションの最中に当たるkillが減っている。

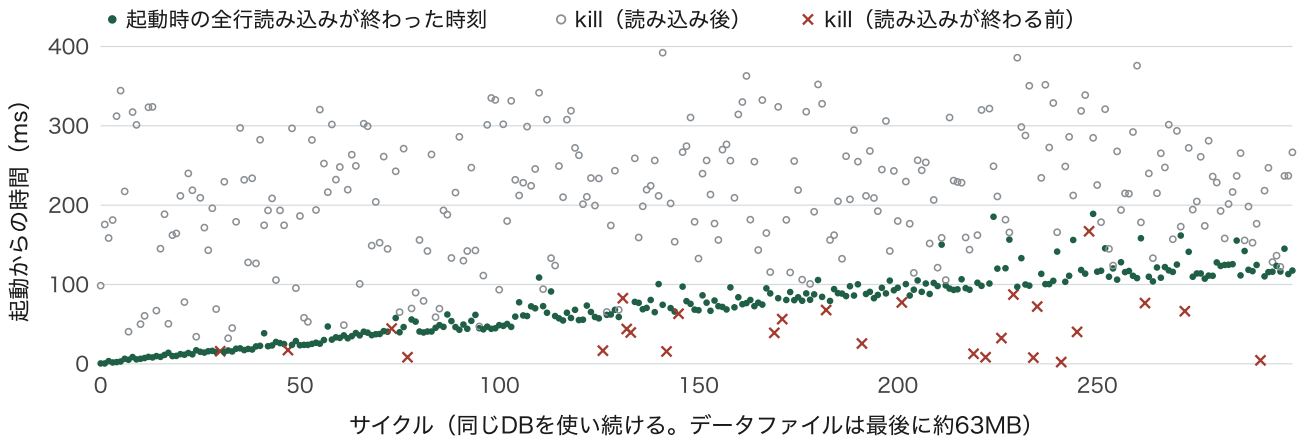


図3　正常版・32枠の300サイクルで、起動時の全行読み込みが終わった時刻（緑の点）とkillの時刻。読み込みが終わる前のkillは赤い×で示した（27回)。サイクルが進むほど読み込みが長くなり、×が増える。

WALの末尾が書きかけのまま残っていたことは、今日行ったすべての実行（合計4,679回のkill）で0回だった。書きかけの末尾を捨てる処理は入っているが、このテストでは一度も動いていない。

テスト側で直した穴

最初のまとめ実行（20:20）でも、正常版はチェック205回と73回ですべて期待どおりで、WAL抜きは39回中38回、復旧飛ばしは49回中46回検出していた。数字だけなら十分に見えたが、その後の調べ直しで、エンジンのバグが1件と、テストの穴がいくつか見つかった。表2はそれらを全部直した後の結果である。

エンジンのバグは、挿入で新しいページを確保するときの順序だった。確保できたかを確かめる前にページ数を増やしていたため、Buffer Poolが埋まって確保に失敗した後もそのトランザクションを続けると、存在しないページを最後のページとして扱ってしまう。当時のテストでは、確保に失敗した後にトラ

ンザクションを続けることがなかったので、表に出ていなかった。再現する単体テストを先に書いて失敗を確かめてから直した。

テストの穴は、大きく分けて三つあった。一つ目は、壊れ版が足りなかったこと。最初はWAL抜きと復旧飛ばししかなく、NO-STEALを破る誤りを見つけられるかは確かめていなかった。Buffer Poolの枠数やチェックポイントの間隔も変えられなかった。そこでコミット前書き込み版、fsync抜き版、8枠の条件を足した。

二つ目は、足したコミット前書き込み版そのものの誤りである。この版は、取り消したトランザクションが新しく作ったページを元に戻していなかった。そのため、killしなくても、取り消した行が開き直したときに残る。小さなプログラムで再現すると、正常版は0行、この版は1行残った。この誤りは21:19と21:40のまとめ実行でも残っていて、そのときのこの版の検出数には、狙っていた「クラッシュしたときだけ見える誤り」とは別の分が混ざっていたことになる。

三つ目は、合否の出し方が甘かったこと。ハーネスがありえない台帳を見つけても集計にも終了コードにも出なかった。まとめ実行は、どれかが失敗しても、壊れ版が何も検出しなくても、成功で終わっていた。中身の薄い実行も合格になった。UBSanは初期設定では問題を報告するだけで止まらないので、検査つきの実行で未定義動作が起きても見落とす可能性があった。どれも「確かめ方」に書いた形に直した。

killのタイミングの説明も直した。それまで「0〜250msのランダムなタイミングでkill」と書いていたが、ハーネスの待ち時間を単独で測ると、指定よりかなり長かった（100msの指定で232〜249ms）。最後のまとめ実行で記録したkill時刻は前に書いたとおりで、延び方は一定ではない。合否には影響しないが、今は実際にkillした時刻を記録している。また、正常版の後半でコミットが1件も完了しないサイクルが出ることに気づいて調べたところ、図3の現象だった。

表3　コミット前書き込み版の検出（チェックのうち食い違いを検出した回数）

まとめ実行	32枠	8枠	この版の状態
21:19	28/48	22/53	取り消しの誤りがある
21:40	12/43	23/44	取り消しの誤りがある
22:29（最後）	16/43	10/44	直した後

表3のとおり、同じ条件でも実行ごとの差が大きい（32枠では28/48と12/43）。枠数による違いも、21:19では32枠のほうが多く、21:40では8枠のほうが多かった。直す前と後の差を、直したことの効果とも、枠数の効果とも言えない。

まだ確かめていないこと

- **電源断。**macOSの `man 2 fsync` には次のようにある。

  > Note that while fsync() will flush all data from the host to the drive (i.e. the "permanent storage device"), the drive itself may not physically write the data to the platters for quite some time and it may be written in an out-of-order sequence.

同じページには、データベースのように書き込みの順序が大事なアプリケーションは F_FULLFSYNC を使うべきだと書かれている。rowdbは素のfsyncしか呼んでいないので、このMacで電源断に耐えるかは、テスト以前に保証されていない。

- **書きかけのデータ。**WALの末尾が書きかけになる場合と、DBを作るときに最初に書くページ（WALを通さない）が書きかけになる場合は、どちらもこのテストで一度も起きていない。後者は、書き込みの再試行の途中でkillされれば kill -9 でも理屈の上では起こりうる。そうなるとDBを開けなくなるが、この対処はM1では見送っている。
- **チェックポイントの途中のkill。**killがチェックポイントの最中（データファイルへの書き出しからWALを空にするまで）に当たったかどうかは記録しておらず、当たった回数はわからない。
- **ハーネス自身の異常の判定。**台帳がありえない形になっていたときに「ハーネスが信用できない」として失敗にする判定は入れたが、実際にその状況を作って動くことを確かめてはいない。
- **1件だけ取り出す処理（get）。**行の位置と長さの範囲チェックがなく、ハーネスからも呼んでいない（単体テストでは呼んでいる）。
- **チェックを飛ばしたサイクルの扱い。**チェックしなかったサイクルの未確定トランザクションは、次のチェックまで持ち越し、その時点でそれぞれ独立に「全部反映」か「全部なし」のどちらかとみなす。そのため、途中のサイクルの復旧で一度反映された行が、次のチェックまでに消えても、後の操作がその行に触れていなければ「全部なし」として合格してしまう。
- **原因のわからない停滞。**コミット前書き込み版・32枠で2回起きた。21:11の実行（テストを直す前の実行なので、表3には入れていない）では、DBを作り直した直後のcycle 30〜55の26サイクルで、ACKされたコミットはcycle 39の1件だけだった。21:40の実行では、cycle 30〜41の約10秒間、復旧・読み込み・コミットが全部遅くなった。その間のチェックは確かめる中身がほとんどない。同じ原因かどうかも含めて、わかっていない。最後のまとめ実行でも、読み込みが終わってから50ms以上ACKが1件も出なかったサイクルが、fsync抜きで1回あった。
- **結果の範囲。**Mac 1台、1スレッド、削除した場所を使い直さない設計での結果である。

次に決めること

次の候補は三つある。上に挙げた小さな残り（getと、チェックを飛ばしたサイクルの扱い）を先に片付けること。インデックス（B+木）を作ること。電源断のテストを作ること。電源断のテストはLinuxの仮想マシンなどが必要で、規模が大きくなる。